Множественное наследование

Множественное наследование

```
class A {
public:
    int x = 2;
};


class B {
public:
    int y = 4;
};


class C: public A,  B {
public:
    int z = 5;
};
```

Расположение в памяти:

x	y	z

```
int main() {
    C c;
    c.x = 3;  //ок
    c.y = 6;  //Ошибка, private наследование
}
```

Пример

```
class A {
public:
    int x = 2;
};
class B {
public:
    int y = 4;
};
class C: public A,  B {
public:
    int z = 5;
};
void f (A a) {}
void f1 (B b) {}
int main() {
    C c;
    c.x = 3;
    f(c);           //Ok
    f1(c);          //Private наследование, запрещено
    A* pa = &c;
    std::cout << pa->x << std::endl;  //3
}
```

Одноимённые методы в базовых классах

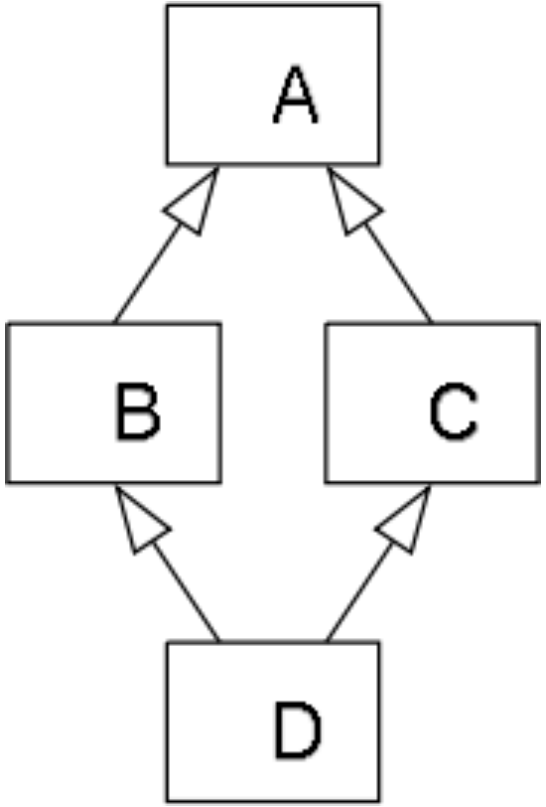
```
class A {
public:
    int x = 2;
    void f() {std::cout << "A\n";}
};
class B {
public:
    int y = 4;
    void f() {std::cout << "B\n";}
};
class C: public A,  public B {
public:
    int z = 5;
};
void f (A a) {}
void f1 (B b) {}
int main() {
    C c;
    c.A::f();
    c.B::f();
}
```

Указатель на метод

```
class A {
public:
    int x = 2;
    void f(){std::cout << "A\n";}
};
class B {
public:
    int y = 4;
    void f() {std::cout << "B\n";}
};
class C: public A,  public B {
public:
    int z = 5;
    void f() { std::cout << "C\n";}
};

int main() {
    void (A::*p)() = &A::f;
    C c;
    (c.*p)();
}
```

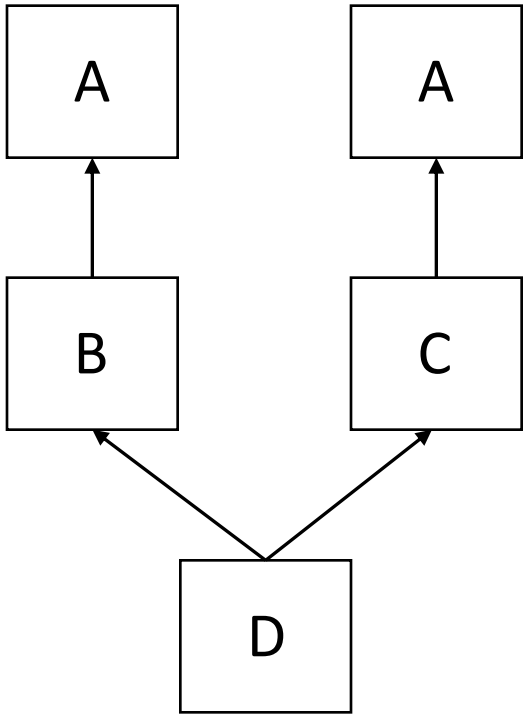
Проблема ромба

```
class A {
public:
    int a = 2;
    void f(){std::cout << ”A\n";}
};
class B: public A {
public:
    int b = 2;
};
class C: public A {
public:
    int c = 4;
};
class D: public B,  public C {
public:
    int d = 5;
};


int main() {
    D d;
    d.f();     //неоднозначность
    d.A::f(); //неоднозначность
}
```



Ромбовидное наследование (продолжение)

```
class A {
public:
    int a = 2;
    void f(){std::cout << "A\n";}
};
class B: public A {
public:
    int b = 2;
};
class C: public A {
public:
    int c = 4;
};
class D: public B,  public C {
public:
    int d = 5;
};
int main() {
    D d;
    d.C::f();  // ok
    A& a = d;  // Ambiguous conversion
}
```



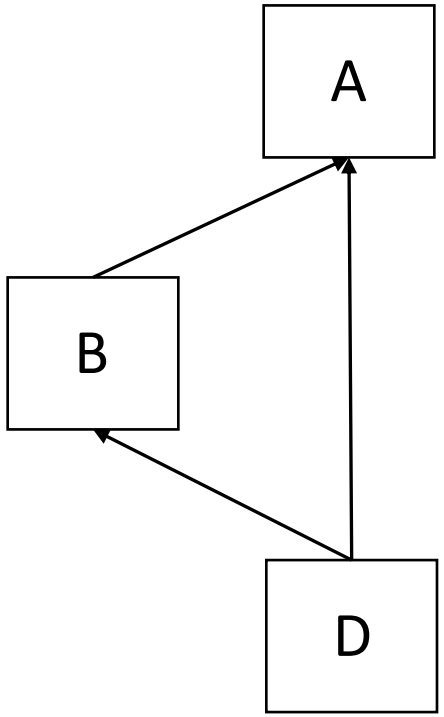
Расположение в памяти:

Пример

```
class A {
public:
    void f(){std::cout << "A\n";}
};
class B: public A {
public:
    void f(){std::cout << "B\n";}
};
class C: public A {
public:
    void f(){std::cout << "C\n";}
};
class D: public B,  public C {
public:
    int d = 5;
    void f(){std::cout << "D\n";}
};

int main() {
    D d;
    d.f();       //D
    d.B::f();    //B
    d.C::f();    //C
    static_cast<B>(d).A::f();   //A
    static_cast<C>(d).A::f();   //A
}
```

Direct base 'A' is inaccessible due to ambiguity

```
class A {
public:
    void f(){std::cout << "A\n";}
};
class B: public A {
public:
    void f(){std::cout << "B\n";}
};
class D: public B,  public A {
public:
    int d = 5;
    void f(){std::cout << "D\n";}
};


int main() {
    D d;
    d.f();       //D
    d.B::f();    //B
    static_cast<B>(d).A::f();   //A
}
```

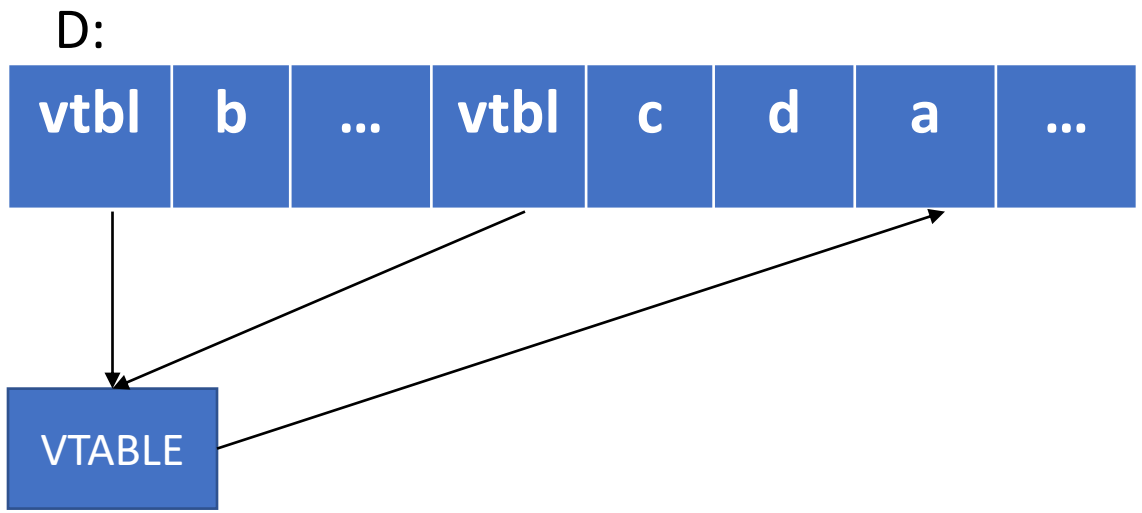


Виртуальное наследование

```
class A {
public:
    void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    void f(){std::cout << "B\n";}
};
class C: public virtual A {
public:
    void f(){std::cout << "C\n";}
};
class D: public B,  public C {
public:
    int d = 5;
};
int main() {
    D d;
    d.A::f();    //A
    A& a = d;
    d.B::f();   //B
    static_cast<B>(d).A::f();   //A  , но d.f() неоднозначно
}
```

Как располагается в памяти?

```
class A {
public:
    int a = 3;
    void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    int b = 4;
    void f(){std::cout << "B\n";}
};
class C: public virtual A {
public:
    int c = 5;
    void f(){std::cout << "C\n";}
};
class D: public B, public C {
public:
    int d = 5;
};
```

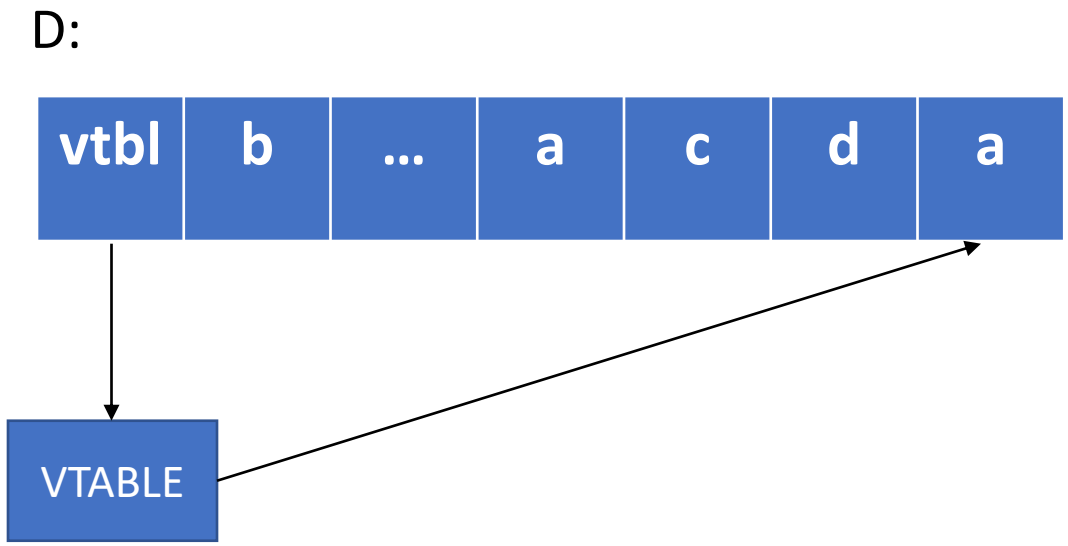


В Vtable перечислены адреса родительских классов

```
int main() {
    D d;
    std::cout << sizeof(d); //40
}
```

Если одно наследование virtual, а другое нет

```
class A {
public:
    int a = 3;
    void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    int b = 4;
    void f(){std::cout << "B\n";}
};
class C: public A {
public:
    int c = 5;
    void f(){std::cout << "C\n";}
};
class D: public B, public C {
public:
    int d = 5;
};
```



Та же проблема ромбовидного наследования

Виртуальность и множественное наследование

```
class A {
public:
    A() = default;
    virtual void print() const { std::cout << "print() from A" << std::endl; }
};
class B {
public:
    B() {}
    void print() const { std::cout << "print() from B" << std::endl; }
};
class C: public A, public B {
public:
    C(): A(), B() {}
    void print() const { std::cout << "print() from C" << std::endl; }
};
int main() {
    C c;
    A& a = c;
    a.print();   //  print() from C
    B& b = c;
    b.print();   // print() from B
}
```

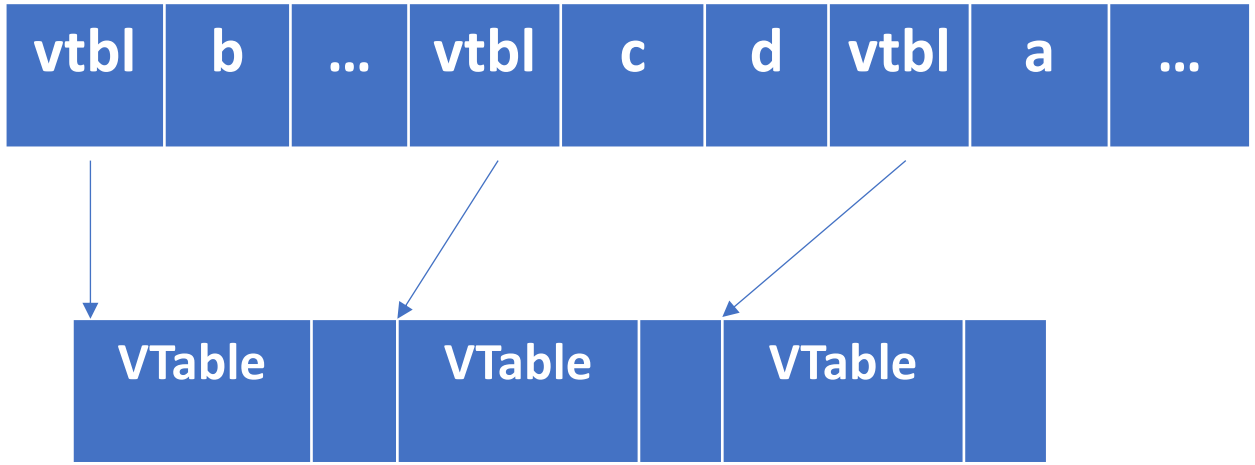
Ромбовидное наследование с виртуальными функциями

```
#include <iostream>
class A {
public:
    int a = 2;
    virtual void f(){std::cout << "A\n";}
};
class B: public A {
public:
    int b = 2;
};
class C: public A {
public:
    int c = 4;
};
class D: public B,  public C {
public:
    int d = 5;
    void f() override {std::cout << "D\n";}
};

int main() {
    A* pa;
    D d;
    pa = &d;
    pa ->f();    // Ambiguous conversion from derived class 'D' to base class 'A':
                // class D -> class B -> class A
}
```

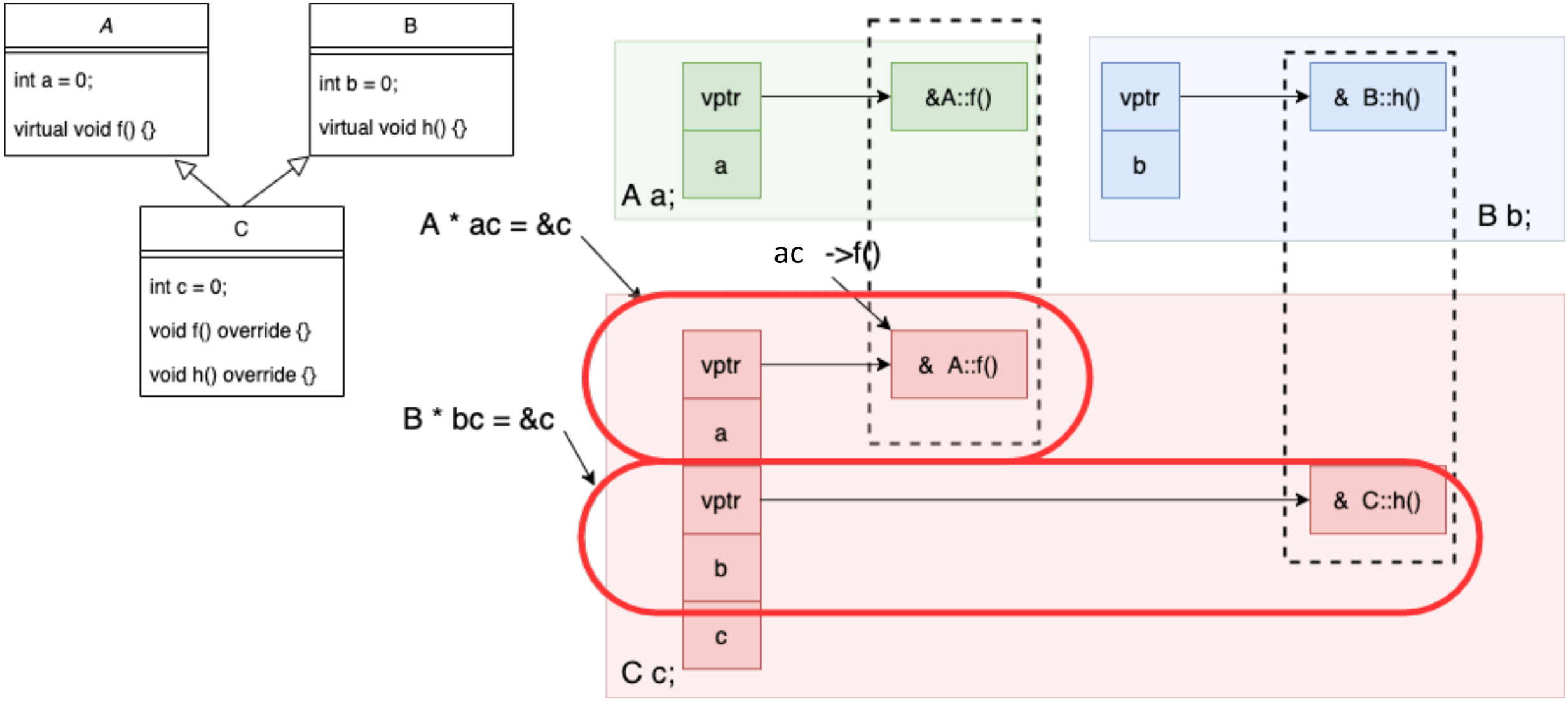
Виртуальное наследование с виртуальными функциями

```
class A {
public:
    int a = 3;
    virtual void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    int b = 4;
};
class C: public virtual A {
public:
    int c = 5;
    void f(){std::cout << "C\n";}
};
class D: public B, public C {
public:
    int d = 5;
};


int main() {
    A* pa;
    D d;
    pa = &d;
    pa->f(); //C
}
```

D d;



Vtable и множественное наследование



Принципы разработки

1. YAGNI

You Aren’t Gonna Need It / Вам это не понадобится

Если пишете **код**, то будьте уверены, что он вам **понадобится**. **Не пишите код** «на всякий случай», думая, что **он пригодится позже**.

Используйте функции git-репозиториев и учитесь фиксировать изменения правильно. После этого не бойтесь удалять лишнее.

2. DRY
Don’t Repeat Yourself / Не повторяйтесь

Дублирование кода – пустая трата времени и ресурсов. Вам придется поддерживать одну и ту же логику и тестировать код сразу в двух местах, причем если вы измените код в одном месте, его нужно будет изменить и в другом.

В большинстве случаев дублирование кода происходит из-за **незнания системы**.

3. KISS
Keep It Simple, Stupid / Будь проще

Не придумывайте к задаче более **сложного решения**, чем ей требуется.

4. Big Design Up Front
Глобальное проектирование прежде всего

Прежде чем переходить к реализации, **убедитесь, что все хорошо продумано**.

5. SOLID
Single-responsibility principle /Принцип единственной ответственности

Каждый объект, класс и метод должны отвечать только за что-то одно. Если ваш объект/класс/метод делает слишком много, вы получите спагетти-код.

Побочный эффект такого кода – проблемы с тестированием. Запутанный функционал тестировать сложно.

6. Avoid Premature Optimization
Избегайте преждевременной оптимизации

Прежде чем вы погрузитесь в детали реализации, убедитесь, что эти оптимизации действительно полезны.

Преждевременная оптимизация может привести к задержкам в коде и, следовательно, увеличит затраты времени на вывод функций на рынок.

7. Бритва Оккама

Бри́тва О́ккама (иногда ле́звие О́ккама) — методологический принцип, в кратком виде гласящий: «Не следует множить сущее без необходимости» (либо «Не следует привлекать новые сущности без крайней на то необходимости»)

-Википедия

Don’t repeat yourself

2. Шаблоны.

    Шаблоны функций

    Шаблоны классов

Классический пример на удобство использования шаблонов

```
void swap (double& a, double& b) {
    double temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    int b = 6;
    swap(a,b);                                  No matching function for call to 'swap'
    std::cout << a << b;

}
```

Следовательно, следует перегрузить функцию для другого типа данных. Но тело функции особо не изменяется:

```
void swap (double& a, double& b) {
    double temp = a;
    a = b;
    b = temp;
}
void swap (int&a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

template <typename T>

Здесь T – это параметр шаблона

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    int b = 6;
    swap(a,b);
    std::cout << a << b;
}
```

Несколько определений

- Обобщённое описание функции называется **шаблоном функции**. Шаблон без подстановки конкретного типа не превращается в реальный код.
- Сгенерированную конкретную функцию по шаблону называют **шаблонной функцией**.
- **Парадигму программирования**, в которой единожды описанный алгоритм может применяться для разных типов, называют **обобщённым программированием**.
- Под термином **метапрограммирование** объединяют техники написания кода, который генерирует новый код в результате исполнения.

Инстанцирование

(англ. instance — создание экземпляра чего-то) — **процесс порождения специализации**. Компилятор или разработчик где-то порождает код через подстановку параметров в шаблон, инстанцирование может быть неявным и явным.

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    int b = 6;
    swap(a,b);
    std::cout << a << b;
}
```

Пример неявного инстанцирования

Неявное инстанцирование не работает для неявного преобразования типов

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    double b = 6;
    swap(a,b);  //ошибка!
    std::cout << a << b;
}
```

Явное инстанцирование

```
template <typename Type>
Type max(Type a, Type b) {
  return (a >= b ? a : b);
}
int main () {
    int a = 5;
    double b = 6;
    int c = max<int>(a,b);
    std::cout << a << b << c;
}
```

Преобразуется к max<int>(int, int)

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    double b = 6;
    swap<double>(a,b);
    std::cout << a << b;
}
```

Candidate function template not viable: no known conversion from 'int' to 'double &' for 1st argument

Если в функции ожидаются разные типы, можно сделать так:

```
template <typename T, typename U>
T& sum(T& a, U& b) {
    a+=b;
    return a;
}
int main () {
    int a = 8;
    double b = 5.7;
    std::cout << sum<int,double>(a, b);
}
```

Или даже так:

```
std::cout << sum(a, b);
```

Deduce. Компилятор умеет сам выводить тип

```
template <typename T, typename U>
T& sum(T& a, U& b) {
    a+=b;
    return a;
}
int main () {
    int a = 8;
    double b = 5.7;
    std::cout << sum(a, b);
}
```

При однозначном выведении типа можно его вообще не указывать либо указать частично.

```
template <typename T, typename U>
const T& min(const T& a, const U& b) {
    return (a < b)?a:b;
}
int main () {
    int a = 8;
    double b = 5.7;
    std::cout << min(a, b);
}
```

Функция min возвращает значение, и невозможно выбрать, какой тип из двух следует вернуть (T или U)?

Таким образом, при проектировании становится важным порядок расположения шаблонных аргументов. Те, которые компилятор может вывести, стоит располагать в конце списка.

Возвращаемый тип должен быть явно задан при вызове

Если тип возвращаемого значения шаблона функции является независимым параметром шаблона, он всегда **должен явно задаваться при вызове**, поскольку его невозможно вычислить на основании аргументов.

```
template<typename T, typename U>
U f(T a) {
    return a;
}
int main () {
    int a = 8;
    std::cout << f<int>(a);
}
```

No matching function for call to 'f'

Перегрузка шаблонов функций

Одноименные шаблоны функций могут перегружаться так же, как и обычные функции. Обрабатывая вызов функции в программе, компилятор должен решить, шаблон или обычная функция лучше всего подходит для данного вызова.

```
template <typename T>
const T& min(const T& a, const T& b) {
    return (a<b) ? a : b;
}
const char* min(const char* a, const char* b){
    return (std::strcmp(a,b) < 0) ? a: b;
}
double min(double x, double y) {
    return (x < y) ? x : y;
}
int main () {
    const char *s2 = "Hello";
    const char *s1 = "Good-bye";
    std::cout << min(1, 2) << std::endl;    //шаблон
    std::cout << min(1.0, 2.0) << std::endl; //double
    std::cout << min(1, 2.0) << std::endl;  //double
    std::cout << min(s1, s2) << std::endl;  //const char*
    std::cout << min<>(s1, s2) << std::endl; //шаблон
}
```

<> говорит компилятору, что вы явно хотите использовать шаблон

Пояснения к предыдущему слайду

Если бы шаблона не было, то для первой строки была бы вызвана double-версия min(), так как существует стандартное преобразование int в double. Однако шаблон может сгенерировать int-версию, которая обеспечивает лучшее совпадение; именно это и происходит в программе. Вызов во второй строке точно совпадает с double-версией, а в третьей строке снова вызывается та же функция, при этом 1 автоматически преобразуется в 1.0. В четвертой строке напрямую вызывается версия min()для const char*. В строке 5 мы заставляем компилятор задействовать шаблон, присоединяя пустую пару угловых скобок к имени функции. Компилятор генерирует специализацию для const char* и использует ее (в этом можно убедиться по неправильному ответу - компилятор просто сравнивает адреса.

Приоритеты шаблонов

```
template <typename T>
void f(T);

template <typename T>
void f(T*);

template <typename T>
void f(const T*);
```

Для первого шаблона подходит любой тип. Второй шаблон более специализирован, чем первый, потому что для него подходят только типы-указатели. Третий шаблон может вызываться только для указателей на const, тогда как второй шаблон допускает любые типы указателей.

?

```
template <typename T>
void f(T& a) {
    std::cout << "T& \n";
}
template <typename T>
void f(T a) {
    std::cout << "T \n";
}


int main () {
    int x = 3;
    f(x);  //ошибка, обе подходят
    f(2); // вторая
}
```

```
template <typename T>
void f(const T& a) {
    std::cout << ”const T& \n";
}
template <typename T>
void f(T& a) {
    std::cout << "T& \n";
}


int main () {
    int x = 3;
    f(x); //вторая
    f(3);  //первая
}
```

Специализация шаблона

Специализация шаблона функции — это механизм, позволяющий определить особую реализацию шаблона для конкретных типов данных, отличную от общего (основного) шаблона.

```
template <typename T, typename U>
void f(T, U) {
    std::cout << 1;
}
template <> //специализация
void f(const char*, double) {
    std::cout << 3;
}
template <> //специализация
void f(char*, double) {
    std::cout << 4;
}
int main () {
    f("test", 5.0); //3
    f(1,4);         //1
    char* str;
    f(str, 4.8);    //4
    f(str, 4);      //1
}
```

Специализация шаблона функции

Например, такая исходная функция:

```
template <typename T>
const T& min (const T& a, const T& b) {
    return (a < b) ? a : b;
}
```

Если эту шаблонную функцию специализировать для char*:

```
template<>
char* const& min<char*> (char* const& a, char* const& b) {
    std::cout << "Work again\n";
    return (strcmp(a,b)<0)?a : b;
}
```

Если эту шаблонную функцию специализировать для const char*:

```
template<>
const char* const& min<const char*> (const char* const& a, const char* const& b) {
    std::cout << "Work again\n";
    return (strcmp(a,b)<0)?a : b;
}
```

Потому что const T& — это **константная ссылка на T**. Следовательно, при «подстановке» char*, константность применяется ко всему char*, получается char* const&, при инстанцировании const char* - получается const char* const&

Отличие const char*& от char* const&

```
char test[25]{"test"}, ou[3]{"ou"};
char* const& p = test;
//p = ou; нельзя менять константу
p[0] = 'f'; //но можно менять то, что под указателем
std::cout << p; //fest
```

```
const char *test = "test";
const char *hello = "hello";
const char*& p = test;
//p[0] = 'f'; нельзя менять то, что под указателем
std::cout << p; //test
p = hello; //можно менять сам указатель
std::cout << p; //hello
```

Специализация шаблона функции

```
template <typename T>
const T& min (const T& a, const T& b) {
    return (a < b) ? a : b;
}
template<>
const char* const& min<const char*> (const char* const& a, const char* const& b) {
    std::cout << "Work again\n";
    return (strcmp(a,b)<0)?a : b;
}
const char* min(const char* a, const char* b) {
    std::cout << "Work\n";
    return (strcmp(a,b)<0)?a : b;
}

int main () {
    const char* s2 = "Hello! ";
    const char* s1 = "Hi";
    std::cout << min(s1,s2) << '\n’; //Work
    std::cout << min<>(s1,s2) << '\n’; //Work again
}
```

При работе с шаблонными функциями рекомендуется использовать перегрузку функции вместо специализации шаблона

Перегрузка и специализация шаблона функции – это разные вещи

```
template <typename T, typename U>
void f(T, U) {
    std::cout << 1;
}
template <>
void f(int, int) {
    std::cout << 3;
}
template <typename T>
void f(T, T) {
    std::cout << 2;
}


int main () {
    f(0,0); //2
}
```

```
template <typename T, typename U>
void f(T, U) {
    std::cout << 1;
}
template <typename T>
void f(T, T) {
    std::cout << 2;
}
template <>
void f(int, int) {
    std::cout << 3;
}


int main () {
    f(0,0);   //3
}
```

Любая специализация шаблона функции будет восприниматься как перегрузка шаблона.

Специализация идёт сразу после шаблона.

Частичная специализация в функциях запрещена

Частичная специализация позволяет определить **особую версию шаблона для группы типов**, а не для одного конкретного типа

```
template <typename T, typename U>
void f(T, U) {
    std::cout << 1;
}
template <typename T>
void f<T,int>(T, int) {
    std::cout << 3;
}
```

Function template partial specialization is not allowed

Выводы

1. Шаблонные функции не имеют реального кода, пока не появится их явный вызов с инстанцированием. Всё написанное можно воспринимать как «псевдокод».
2. Поэтому компилятору необходимо уметь точно определять тип возвращаемого параметра, остальные типы он может иногда вывести сам.
3. Если есть нешаблонная функция, которая будет оптимально подходить по передаваемым параметрам, выберется именно она, если не указать явное требование использовать шаблон.
4. Можно использовать специализацию шаблонной функции, но лучше пользоваться простой перегрузкой.
5. Специализация пишется сразу после шаблона.

Шаблон класса

```
template <typename T>
class vector {
    T* arr;
    size_t sz;
    size_t cap;
};
```

При объявлении генерируются разные классы для разных типов

```
int main () {
    vector<int> A;
    vector<char> B;
    vector<double> C = A;  //Будет ошибка
}
```

Нетиповой параметр шаблона (у функций также может использоваться)

Нетиповой параметр шаблона должен быть целочисленным значением, известным на стадии компиляции. Начиная с C++20 не только целочисленным.

```
template <typename T, size_t N>
class array {
    T arr[N];
};


int main () {
    array<int, 20> A;
    array<char, 100> B;
}
```